



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 40

Dr. Asad Mahmood
May 07th, 2026

Lecture Outline

- Announcements (Quiz Solutions, Final Exam Preparations)
- Outline of Previous Lecture:
 - **Chapter 5 – Large and Fast: Exploiting Memory Hierarchy**
 - 5.4 Measuring and Improving Cache Performance
- Outline of Today's Lecture:
 - **Chapter 5 – Large and Fast: Exploiting Memory Hierarchy**
 - 5.4 Measuring and Improving Cache Performance

Chapter 5

Large and Fast: Exploiting Memory Hierarchy

Chapter Outline

Chapter 5 – Large and Fast: Exploiting Memory Hierarchy

- 5.1 Introduction
- 5.2 Memory Technologies
- 5.3 The Basics of Caches
- 5.4 Measuring and Improving Cache Performance
- 5.5 Dependable Memory Hierarchy
- 5.6 Virtual Machines
- 5.7 Virtual Memory
- 5.8 A Common Framework for Memory Hierarchy
- 5.6 onwards (self-reading)

Handling Writes

- **Solution 3: Write-Back:**
 - On data-write hit, just update the block in cache
 - Declare that block 'dirty' and keep track of whether or not a block is dirty
 - When a dirty block is replaced
 - Write it back to memory
 - Can use a write buffer to allow replacing block to be read first
 - Pros/Cons ?

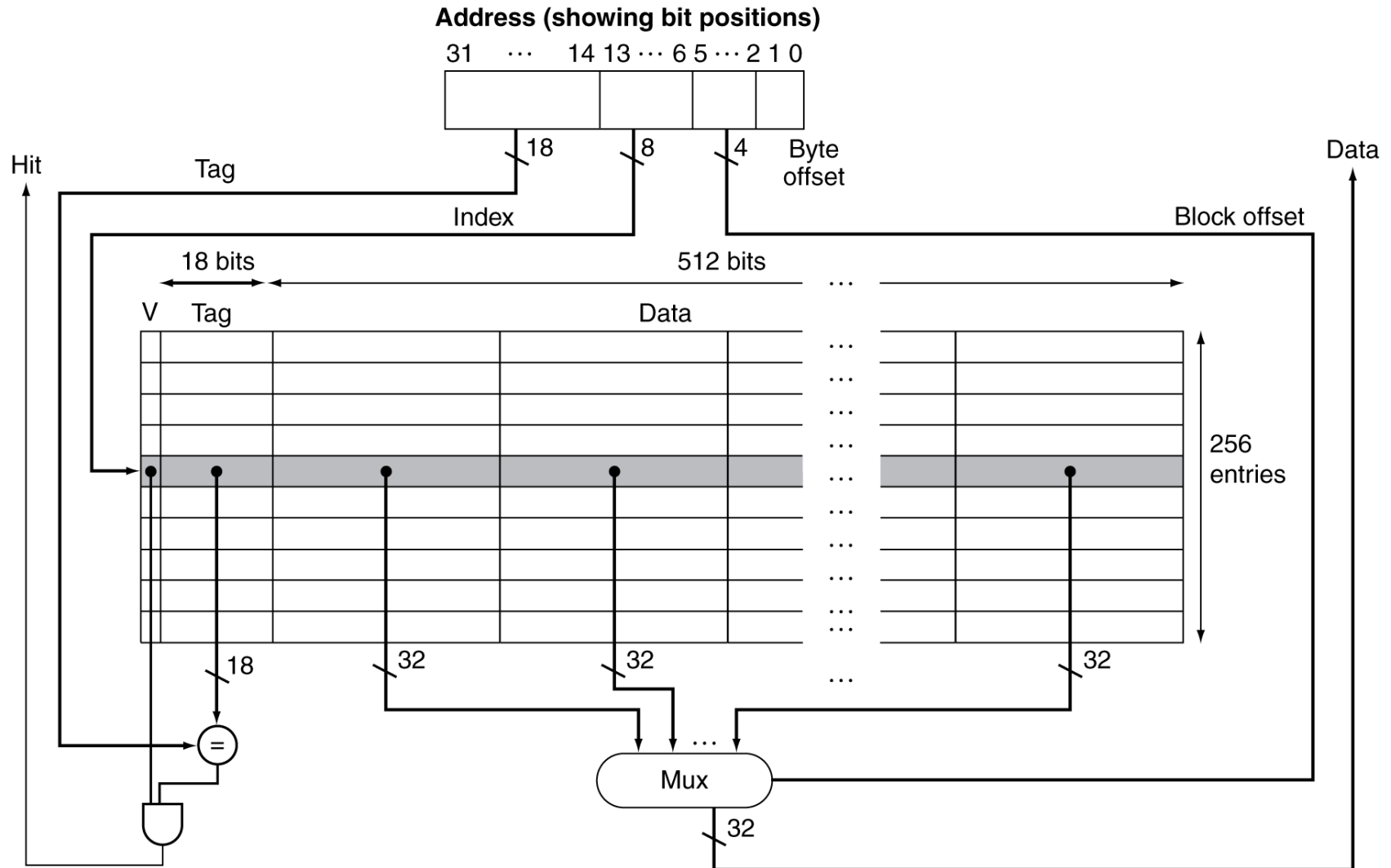
Handling Writes

- What should happen on a *write miss*?
 - **Solution 1: Write Allocate:**
 - *Fetch* the block
 - Perform *write* in cache
 - Usually paired with write-back cache
 - Works best when data is updated frequently
 - **Solution 2: No-Write-Allocate (Write around):**
 - Data written directly into memory
 - Usually paired with write-through policy
 - Works best when data is used once and rarely reused

Example: Intrinsicity FastMATH

- Embedded MIPS processor
 - 12-stage pipeline
 - Instruction and data access on each cycle
- Split cache: separate I-cache and D-cache
 - Each 16KB: 256 blocks × 16 words/block
 - D-cache: write-through or write-back
- SPEC2000 miss rates
 - I-cache: 0.4%
 - D-cache: 11.4%
 - Weighted average: 3.2%

Example: Intrinsity FastMATH



Chapter Outline

Chapter 5 – Large and Fast: Exploiting Memory Hierarchy

- 5.1 Introduction
- 5.2 Memory Technologies
- 5.3 The Basics of Caches
- 5.4 Measuring and Improving Cache Performance
- 5.5 Dependable Memory Hierarchy
- 5.6 Virtual Machines
- 5.7 Virtual Memory
- 5.8 A Common Framework for Memory Hierarchy
- 5.9 onwards (self-reading)

Measuring Cache Performance

- Let us try to quantify cache performance with simplifying assumptions
- Components of CPU time
 1. Program execution cycles
 - Includes cache hit time
 2. Memory stall cycles
 - Mainly from cache misses
 - Read-stall cycles
 - Write-stall cycles (more complicated than read stalls e.g. if write buffer also involved, it can induce stalls as well!)

Cache Performance Example

- Given
 - I-cache miss rate = 2%
 - D-cache miss rate = 4%
 - Miss penalty = 100 cycles
 - Base CPI (ideal cache) = 2
 - Load & stores are 36% of instructions
- Analyze the impact of cache misses on CPU performance vis-à-vis a perfect cache (one that has no misses/stalls)
- What happens if one only improves the CPU performance (e.g. CPI made 1 via improved pipelining) but nothing is done about memory

Average Access Time

- Hit time is **also important** for performance
 - Many possible reasons e.g. increase in cache size
- Average memory access time (AMAT)
- Example
 - Given:
 - CPU with 1ns clock,
 - hit time = 1 cycle,
 - miss penalty = 20 cycles,
 - I-cache miss rate = 5%
 - Find AMAT

Performance Summary

- When CPU performance increased **miss penalty becomes more significant**
 - E.g. Decreasing base CPI or increasing clock-rate results in greater proportion of time spent on memory stalls
- **Conclusion: Can't neglect cache behavior when evaluating system performance!**

Reducing Cache Misses via Flexible Block Placement

Associative Caches

■ Fully associative

- A given block can go in any cache index/entry/block/line
- Requires all entries to be searched at once
- Comparisons need to be done for all entries (expensive)

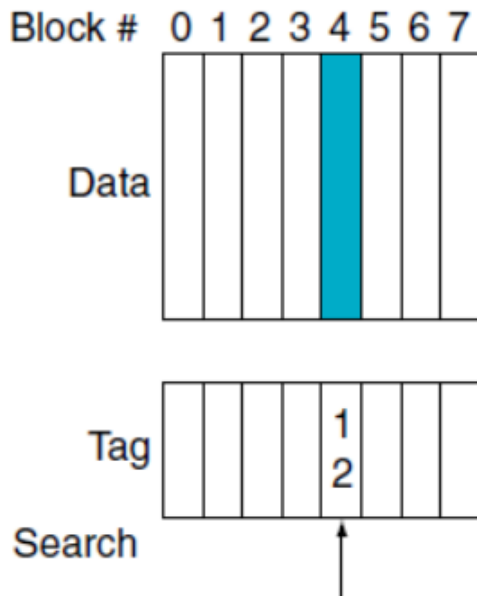
■ *n*-way set associative

- Cache divided into sets, with *n* entries/blocks per set
- A given block can go in any location/entry/block within a SET.
- Search all entries in a given set at once
- *n* comparators needed (less expensive)
- Set number can be determined from block address/number

Associative Cache Example

- Cache size = 8 blocks
- Desired block memory address = 12
- Cache location ?

Direct mapped



Set associative



Fully associative



Spectrum of Associativity

- We can also think of all block placement strategies as a variation on set associativity.
- Consider a 8-block cache
- One-way set-associative cache
- Two-way set-associative cache
- Four-way set-associative cache
- Eight-way set-associative cache

One-way set associative
(direct mapped)

Set	Tag	Data
0		
1		
2		
3		
4		
5		
6		
7		

Two-way set associative

Set	Tag	Data	Tag	Data
0				
1				
2				
3				

Four-way set associative

Set	Tag	Data	Tag	Data	Tag	Data	Tag	Data
0								
1								

Eight-way set associative (fully associative)

Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data

Associativity Example

- Compare three small caches, each consisting of four one-word blocks
 - Direct mapped, 2-way set associative, fully associative
 - Block access sequence: 0, 8, 0, 6, 8
- Direct mapped

Block address	Cache index	Hit/miss	Cache content after access			
			0	1	2	3
0	0	miss	Mem[0]			
8	0	miss	Mem[8]			
0	0	miss	Mem[0]			
6	2	miss	Mem[0]		Mem[6]	
8	0	miss	Mem[8]		Mem[6]	

Associativity Example

■ 2-way set associative

Block address	Cache index	Hit/miss	Cache content after access			
			Set 0		Set 1	
0	0	miss	Mem[0]			
8	0	miss	Mem[0]	Mem[8]		
0	0	hit	Mem[0]	Mem[8]		
6	0	miss	Mem[0]	Mem[6]		
8	0	miss	Mem[8]	Mem[6]		

■ Fully associative

Block address		Hit/miss	Cache content after access			
0		miss	Mem[0]			
8		miss	Mem[0]	Mem[8]		
0		hit	Mem[0]	Mem[8]		
6		miss	Mem[0]	Mem[8]	Mem[6]	
8		hit	Mem[0]	Mem[8]	Mem[6]	

Associativity Example

- If we had 8 blocks in the cache for previous example, the performance of 2-way associative and fully-associative would have been the same!
- If we had 16 blocks in the cache for previous example, the performance of all three placement strategies (direct-mapped, 2-way associative, fully-associative) would have been the same!
- Conclusion: Cache size and associativity are not independent in determining cache performance.